Breaking the Validator Stall Loop

Cost-accounted order-then-execute for F1R3Node:
analysis, design, and development plan

F1R3FLY.io — design document, revision 2

October 16, 2026

Repository	`F1R3FLY-io/f1r3node-rust`
Stall review basis	`master` at `7264c98` (Oct 15, 2026)
Branch reviewed	`feature/cost-accounted-rho` at `ccbee928` (Oct 13, 2026)
Integration target	`dev` at `555a5e44` (Oct 16, 2026)
Revision 2	Incorporates the Casper Working Group response to §5.10.1 and the status of delivered work (Oct 16, 2026)
Method	Code reading with line-level verification. No profiling was run for this document.

Abstract

Validators stall as they come under sustained load. A code review traces the stall to a set of feedback loops that all feed on one quantity: the finality lag, the distance between the tips of the block DAG and the last finalized block (LFB). Many per-block and per-proposal costs grow with that lag, which slows proposals and validation, which lengthens the lag. Two hard limits then turn the slowdown into a stop. The root of most of this cost is run-to-completion execution: because each deploy runs to completion against a committed state, concurrently produced blocks hold divergent post-states, and the node must merge them. The cost-accounting design removes the reason for the merge. Every potential conflict is marked by signature, so deploys can run at any time, and agreement reduces to two things: the state changes of each block, settled by one runner and checked by replay, and the order of conflicting blocks. This document reports the stall analysis, assesses how much of it the `feature/cost-accounted-rho` branch addresses today, gives a detailed design for cost-accounted order-then-execute on v6 shards, and lays out a development plan that lands it in `dev` with minimal change to the legacy path. Revision 2 replaces the finalizer-progress rule of revision 1, which was unsafe, with a design that separates a resumable computation, a validation under the commit-time snapshot, and an atomic commit (§5.11). It also records the work delivered so far.

Contents

1 Summary **4**

2 The stall: analysis **5**

2.1 The core loop 5

2.2 Costs that grow with the lag . . . 6
2.3 Bottlenecks and starvation . . . 7
2.4 Two defects independent of load . . . 8
2.5 Soak evidence . . . 8

3 Why the merge exists, and how cost accounting removes it 8
3.1 Run-to-completion . . . 8
3.2 Conflicts marked by signature . . . 8
3.3 Agreement reduces to two things . . . 9

4 The branch as it stands 9
4.1 Effect on the loop . . . 9
4.2 The branch's own order-then-execute proposal . . . 10

5 Design: cost-accounted order-then-execute 11
5.1 Scope and principles . . . 11
5.2 Block content . . . 11
5.3 Structural validity . . . 11
5.4 Canonical order . . . 12
5.5 The executor . . . 12
5.6 The order-independent charge model . . . 13
5.7 Records, canonical records and attestation . . . 14
5.8 The inclusion filter . . . 15
5.9 Validator sets, epochs and slashing . . . 15
5.10 Backpressure and finality . . . 15
5.11 Finalizer forward progress . . . 15
5.12 Signature footprints and the partial-order refinement . . . 18
5.13 How the design addresses the loop . . . 19
5.14 Properties to verify . . . 19

6 Development plan 20
6.1 Status (October 16, 2026) . . . 20
6.2 Principles . . . 21
6.3 What is kept, ported, and left behind . . . 21
6.4 Hook points in the Casper code of dev . . . 22
6.5 Track A: stall fixes on dev . . . 23
6.6 Track B: cost-accounted order-then-execute . . . 24
6.7 Sequencing . . . 25

6.8 Ownership 26

7 Risks and open questions **26**

7.1 Risks 26

7.2 Open questions for the Casper team 27

A Detailed stall findings **28**

B Sources **29**

1 Summary

The problem. Under sustained deploy load, the unfinalized region of the DAG grows. Proposal preparation, merging, the deploy lifecycle register and the finality oracle all do work that grows with that region. Their combined cost delays blocks and finality, which grows the region further. Two limits close the trap: the finalizer discards any run that takes longer than 15 seconds, and merges refuse outright once the floor distance exceeds 256 blocks. Separately, consensus CPU work runs on the same asynchronous worker threads as networking, so a busy node also stops answering its peers. Section 2 gives the evidence.

The cause underneath. In `dev`, each deploy runs to completion; the only concurrency is the parallel composition inside a deploy. A block's post-state is therefore a function of one linear sequence of deploys. Two concurrent blocks start from a common pre-state and end in divergent post-states, so something must reconcile them. That reconciliation is the merge, and the merge is the largest consumer of the costs that grow with the lag.

The resolution. Under cost accounting, every potential conflict is marked by signature, including races on shared data channels between different signers. Deploys can then run at any time and interleave freely. All conflict resolution moves to the block level, where validators need to agree only on (i) the state changes of each block, which decide the winners of every race, and (ii) the order of conflicting blocks. The first is settled by one runner and verified by replay; the second, in a first version, by a total order on finalized blocks. No sequential scheduler is involved.

The branch today. `feature/cost-accounted-rho` implements the cost-accounting core and a funding gate, but its Casper pipeline is still execute-then-merge. Its interim v6 merge rule removes the exponential conflict search on v6 shards. Otherwise the stall loop is untouched, a v6 block carries at most one user deploy, and the reducer changes cost legacy shards 15–23 % more CPU on the branch's own profiles. The branch's own design proposal (its §7) describes order-then-execute but keeps deploy-at-a-time execution and a single-order scheduler. This document corrects both.

Finality progress. Revision 1 said that a finalizer run that reaches its deadline "adopts the highest candidate proven so far". The Casper Working Group showed that this rule is not safe and gives no progress argument (§5.11). An oracle verdict on one candidate is not a finality decision. Revision 2 adopts the group's replacement: a resumable *Compute* phase, a *Validate* phase under the snapshot at commit time, and an atomic *Commit*. A deadline never cancels Validate or Commit, and no partial result is ever committed. The delivered fix A1a was already consistent with this. It keeps only facts about immutable blocks across runs and commits only complete derivations.

The plan. Two tracks run in parallel (Section 6). Track A fixes the stall on `dev` now, independent of cost accounting, and those fixes are prerequisites for v6 as well, because the finality half of the loop survives any execution model. Track B lands cost accounting in `dev` in seven increments. The legacy path stays byte-identical to `dev`. The interim merge rule, the one-offer block lane and the scheduler are left behind. The v6 consensus seam enters through four hook points behind a genesis-policy switch. Track A items A1a, A2 and A3 and the first v6 tranche are delivered (§6.1).

Decisions recorded for this design (October 16, 2026).

(1) The single-order reduction scheduler is superseded by an order-independent charge model. The scheduler-based decision records are superseded or amended accordingly (§5.6).

(2) Signature marking covers data-channel races between different signers. Two blocks conflict only if their signature footprints intersect (§5.12).

(3) A first version uses a total order on finalized blocks; exploiting the partial order is a later refinement.

(4) The finalizer adopts a candidate only after a complete derivation and a re-validation under the commit-time snapshot. The commit is one atomic transaction. This supersedes §5.10.1 of revision 1 (§5.11).

2 The stall: analysis

2.1 The core loop

Under sustained load three quantities grow together: the pending deploy pool, the set of deploys included in blocks but not yet settled, and the unfinalized DAG cone. Each grows with the finality lag D (tip height minus LFB height), and each carries costs that scale with it (Figure 1).

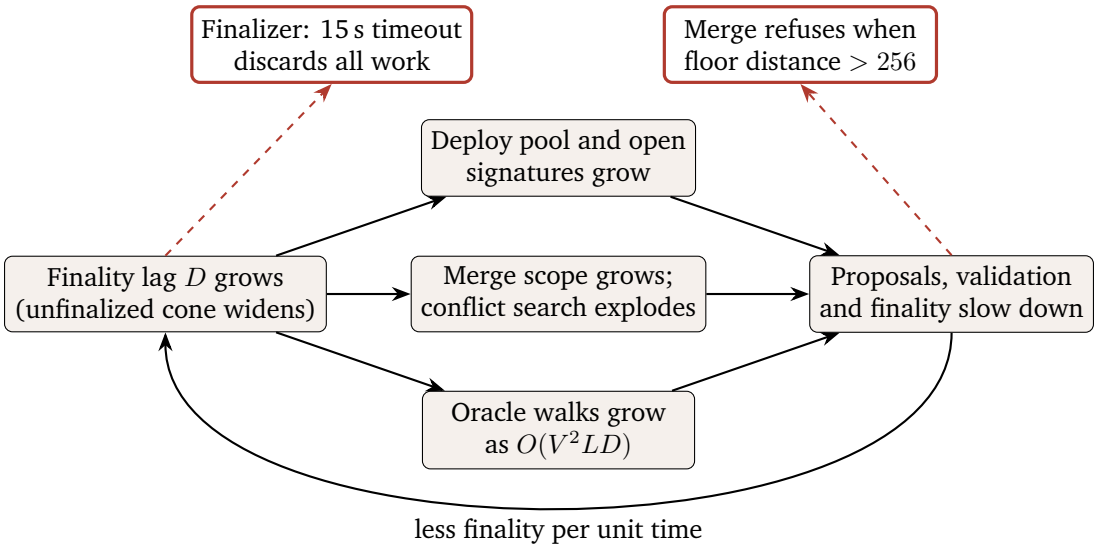


Figure 1: The feedback loop. Every cost on the middle column grows with the finality lag; the two traps (outlined) turn the slowdown into a stop.

Two limits turn the slowdown into a stall:

1. **The finalizer timeout.** `finalization_runner.rs:278–293` wraps each finalizer run in a 15-second `tokio::time::timeout`. On timeout the run is dropped and nothing is adopted. Each later run starts from scratch over a larger cone, so once one run exceeds 15 s the LFB never advances again. A non-zero `finalizer.run.timeouts` counter marks entry into

this regime.

2. **The floor-distance cap.** `interpreter_util.rs:956` sets `MAX_FLOOR_DISTANCE_BLOCKS=256`. Beyond it, `compute_parents_post_state` refuses: proposers park and validators reject. If every validator crosses the cap, the shard can deadlock, because advancing finality may itself need new blocks.

2.2 Costs that grow with the lag

Each item below was verified at the cited lines. Complexity is stated in the quantity that grows under load.

Mechanism	Where and what	Growth
Deploy pool re-read under a lock	`block_creator.rs:590` takes the pool's `parking_lot::Mutex` and holds it to about line 1495. Proposal preparation reaches `read_all()` about four times over the pool and the rejected buffer. Decoding goes through `Signed::from_signed_data` (`casper_message.rs:1363`), which re-verifies every signature.	About $8N$ signature checks per proposal, N the pool size. N grows with lag because included-but-unsettled deploys stay in the pool by design (`block_admission.rs:169`). No admission cap.
Lineage walks per deploy	`effect_in_state_of` (`deploy_lifecycle.rs:93-155`) calls `block_store.get`, which decodes and verifies every deploy of each block, once per selected deploy, per buffered deploy, and per open signature on each floor advance.	Open signatures $\times$ segment length $\times$ deploys per block: quadratic in load.
Rejection-option search	`compute_rejection_options` (`merging_logic.rs:1270-1407`): breadth-first over keep/reject choices, cloning the conflict map per option; duplicate check is a linear scan (`:1369`). Components sharing any mergeable channel are unioned (`conflict_set_merger.rs:905-917`), and the search is re-run until a fixed point (`dag_merger.rs:705-752`).	Exponential in conflicting pairs per group; vault and PoS channels collapse groups into one. Runs on both proposal and validation.

Mechanism	Where and what	Growth
Hot channels in RSpace	Data values are plain `Vec<Datum>` in `imbl` shards (`hot_store.rs:402`): deep-copied on first write after each soft checkpoint, cloned on read, shifted by `insert(0, ..)`, and re-serialized at checkpoint. Replay clones per candidate and walks candidates in $O(K^2)$ (`replay_rspace.rs: 799, 950, 1117–1145`).	$O(n)$ per touch, $O(n^2)$ over a run for a channel accumulating n datums.
Finality walks	Every lookup is an LMDB read plus a full metadata decode; `is_dag_ancestor` is a fresh BFS, called $O(P^2)$–$O(P^3)$ times per floor derivation; the clique oracle makes a new run cache per call (`clique_oracle.rs:793`).	$O(P \cdot V^2 LD)$ warm, $O(D^2)$ cold, per block.
Full table scans under the DAG lock	`get_representation_internal` calls `to_map()` on latest messages and invalid blocks (`block_dag_key_value_ storage.rs:1199–1207`); invalid blocks are never deleted. A read-only equivocation scan takes the global *write* lock (`:1570`) twice per block.	Grows with history; serializes readers including the proposer's snapshot.

2.3 Bottlenecks and starvation

- **Proposer queue.** With auto-propose on, every deploy spawns its own propose request with up to four retries (`block_api.rs:499–515`) into a 1,024-entry queue drained serially, 250 ms plus a full proposal per entry. The "already in progress" branch is unreachable, because the consumer awaits the proposal inline (`proposer_instance.rs:181`). Deploy-triggered requests run in the heartbeat lane, so they can mint empty blocks that every peer must replay.
- **Synchronous locks in async code.** Concurrent `do_deploy` calls park worker threads on the pool mutex while a proposal holds it. When waiting deploys equal the worker count, the runtime, including transport, stops.
- **CPU work on the async runtime.** Replay, `dag_merger::merge`, checkpoints and block creation run on tokio workers without `spawn_blocking`; two parallel validations plus a proposal can pin 3–4 workers and starve gRPC, peer-to-peer traffic and the heartbeat.
- **Heartbeat and polling.** Every accepted block and deploy wakes the heartbeat, whose snapshot cache misses on every insert. gRPC `find_deploy` retries up to 80×100 ms per request, scanning 128 heights per miss.

2.4 Two defects independent of load

- **Buffer eviction removes the newest block.** `casper_buffer_key_value_storage.rs:335-341` computes age as `now-first_seen`, sorts ascending, and removes the first entry, which is the youngest pendant, often the block that just became ready. It also re-sorts the whole buffer per removal.
- **Re-fetch defeats its own rate limit.** `recover_stale_buffer_dependencies` re-requests parents already in the buffer from four peers; the duplicate reply is dropped as `AlreadyProcessed`, and that path clears both the cooldown and the retry counter.

2.5 Soak evidence

The branch's own proposal (`docs/casper/theory/cost-accounting-consensus-proposal.md`, §9.1) records two soaks. In the August soak one merge call averaged 2.16 s, of which the settled-signature probes took 92 %, and the probe depth grew with the finality lag; `dev` has since batched those probes. In the October soak no single stage separated failing sessions from passing ones (AUC at most 0.70), and `dev`'s analysis points to the finality round length and to blocks waiting for missing parents. Read together with the code review, the binding constraint has moved from merge analysis toward finality and Casper bookkeeping. Any design must therefore fix finality as well as remove the merge.

3 Why the merge exists, and how cost accounting removes it

3.1 Run-to-completion

In `dev`, RSpace accepts a deploy, runs it to termination or until its phlogiston is exhausted, commits the state, and only then accepts the next deploy. The only concurrency inside a block is the parallel composition within a deploy. A block's post-state is a function of the parent state and one sequence of deploys.

Validators propose concurrently. Two blocks built on a common ancestor each run their own sequences and end in different post-states. A child with both as parents must combine them: decide which deploys conflict, choose which to reject, and compose the rest. That is the merge, with its event-log indexes, its conflict maps and its rejection-option search. Every block must also be replayed by every validator before acceptance, because its post-state is part of its validity. Merge and replay are the two consensus-path costs that grow with the unfinalized cone.

3.2 Conflicts marked by signature

The cost-accounting papers replace run-to-completion with concurrent acceptance gated by linear resource proofs. Every interaction draws tokens from the signatures involved:

- deploys with *different* signatures draw from disjoint pools and cannot conflict on funding;
- deploys with the *same* signature compete for one pool and are serialized by the linear proof;
- races on shared data channels between different signers are also marked by signature: every communication is charged to each participant's signature, so two deploys that race for the

same datum or continuation both draw on the signature that owns the contested process (decision 2).

The branch's linear cost estimate already counts every participant of a communication: a receive with n binds has $n + 1$ participants (proposal §4.1). The runtime must charge them the same way; §5.12 gives the test. With every potential conflict marked by signature, conflicts become visible at the level of signatures rather than discovered after the fact from event logs.

3.3 Agreement reduces to two things

Deploys can then run at any time and interleave. Nothing requires a block's effect to equal some sequential order of its deploys. Validators need to agree on exactly two things:

1. **The state changes of each block**, which decide, for the whole shard, the winner of every race in it. One runner executes the block and records the outcome; every other node replays the record to verify it.
2. **The order of conflicting blocks.** Blocks whose signature footprints are disjoint commute. A first version simply totally orders all finalized blocks; a refinement orders only conflicting ones.

A sequential scheduler is *not* needed for agreement and is harmful to throughput: it manufactures determinism by discarding concurrency, when agreement only needs the race winners, which a record already carries.

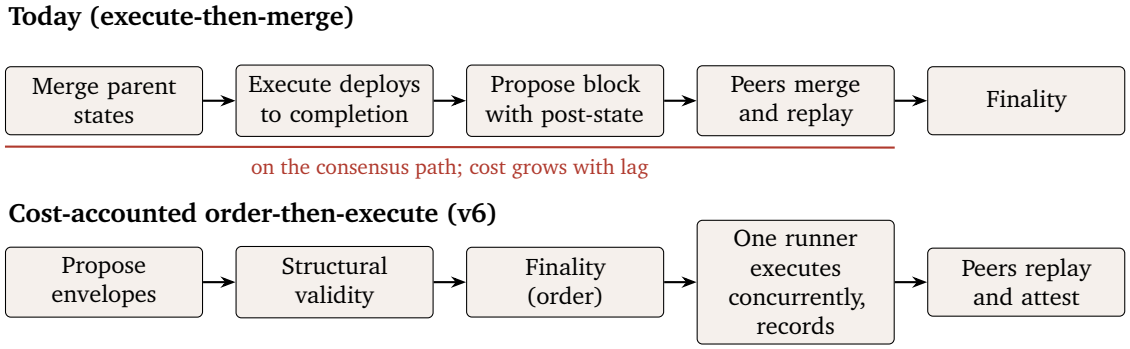


Figure 2: Moving execution behind ordering. In the lower pipeline, consensus never merges or replays.

4 The branch as it stands

`feature/cost-accounted-rho` is 501 commits ahead of its merge base with `dev` (`93e38575`, Oct 6) and 470 behind, about 507,000 added lines. Of those, 184,000 are formal proofs and models, 63,000 design documents, 113,000 the Rholang interpreter (94,000 in `accounting/`), 29,000 RSpace, 18,000 models, and 39,000 Casper (35,000 in the new `costacc/` module). A trial merge into today's `dev` conflicts in 38 files.

4.1 Effect on the loop

Loop component	Effect of the branch
Finalizer timeout, floor-distance cap	Not touched (`floor.rs` $+3/-3$).
Pool re-reads under lock	Not touched; v6 adds offered-envelope scans under the same lock.
Lineage walks, lifecycle register	Not touched.
Rejection-option search	Replaced on v6 shards only by one ordered pass (`dag_merger.rs:1862-1882`, DR-120). The merge index, the larger cost (8–10 % of test work against 1–1.6 % for the merge call), remains.
Hot-channel copies	Priced, not removed: the native replay store charges for copies, and v6 host-work caps bound them. Representation unchanged.
Finality walks, proposer queue, heartbeat, runtime starvation	Not touched.

Two further effects work against throughput:

- **One user deploy per v6 block.** Once a first offered candidate is chosen, every later one becomes a fallback (`block_creator.rs`, about line 1910). Multi-offer blocks (G2-2 onward) are approved but not started. Under load, the pool, and therefore the lag, grows faster on v6 than on legacy.
- **Legacy shards pay for the reducer changes.** The scheduler (`deterministic_reduction.rs`) and the accounting sit inside `ReducerCore` for every shard. The branch's profiles show 15–23 % more CPU than `dev` on legacy multi-parent tests, outside the merge.

4.2 The branch's own order-then-execute proposal

The branch carries a proposal (§7 of the document named above) for order-then-execute on v6 shards. It gets the architecture right in three respects: a canonical total order of finalized blocks along the main chain; one runner with verified replay (since the October 13 amendment); and structural block validity with no merge, execution or replay on the consensus path. It departs from the model of Section 3 in two respects that this design corrects:

1. **Its executor is deploy-at-a-time.** Proposal §7.6 holds, executes and settles one envelope before the next, and §7.10 admits a parallel executor only if its result replays as that sequential batch. That re-imports run-to-completion into the executor.
2. **It keeps the single-order scheduler, or serial replay.** Under native execution every RSpace operation of a deploy joins one conflict component (`deterministic_reduction.rs:1068-1072,1120-1122`) and runs alone. The scheduler exists because some host-work charges read live container sizes; for example, a COMM lookup "is charged at the live sizes of the three identity maps" (`replay_authority.rs`, DR-79). The proposal's exit path (its RD-1) replaces the scheduler with journal-order replay, which moves the serial bottleneck into replay, the work every other validator does.

5 Design: cost-accounted order-then-execute

5.1 Scope and principles

- **v6 only.** The design applies to shards whose adopted genesis resource policy activates offered (cost-accounted) execution. Legacy shards run `dev`'s pipeline unchanged.
- **It remains Casper.** Signed blocks, justifications, LMD-GHOST fork choice, clique-oracle finality and equivocation detection are unchanged. Parents become ordering references, not merge inputs.
- **No sequential scheduler.** Concurrency inside and across deploys is preserved. Agreement comes from records, not from a canonical schedule.
- **One runner, verified replay.** Exactly the pattern by which producers and validators agree today, moved behind finality.
- **Total order first.** Every finalized block is placed in one canonical order. The partial order is a later refinement (§5.12).
- **Bounded lag by construction.** Proposal rate, block demand and execution lag are throttled so that no cost on the consensus path grows with the unfinalized cone (§5.10).

5.2 Block content

A v6 block keeps every consensus field. Its body changes meaning:

Field	Meaning on a v6 shard
`deploys`	Accepted envelopes, in block order, without event logs. Not executed by the proposer.
`state.preStateHash`, `postStateHash`	The attestation (E, r_E): the newest finalized block E whose canonical record the proposer has replayed, and the root after it.
`state.bonds`	The validator set of the block's epoch, taken from attested state (§5.9).
`rejectedDeploys`, `appliedFromScope`, `mergeBase`	Empty. Validity requires them empty.
`systemDeploys`	Slash records to execute at the block's position; close-block work is implicit.
New: `records`	Records produced by this node's executor for finalized blocks of its past cone that no earlier block cites (§5.7). Additive protocol-6 field.

5.3 Structural validity

Validity is a pure function of the block and its past cone. It never depends on how far a node's executor has progressed, and it runs no Rholang and replays nothing.

```
validate_v6(block):
  sender signature, sequence number, justifications, parents     -- unchanged
  equivocation check                                              -- unchanged
  for each envelope e:
      encoding and every signature of e
      validity window of e against the block number
      e not in past(parents)                                      -- repeat rule
  sum(phloLimit) of the envelopes <= per-block demand cap
  rejectedDeploys, appliedFromScope, mergeBase are empty
  for each cited record rec(b): encoding valid, b in past(parents)
  cited records within the per-block record cap
  bonds = epoch_validator_set(parents)          -- may wait for local replay
```

Checks that need executed state (the attestation, the inclusion filter at the attested root, the replay of cited records) are *deferred*: a node runs them when its own replay reaches the relevant block. A deferred failure yields signed slashing evidence; it never invalidates a block after the fact.

5.4 Canonical order

For a block L with main chain $M_0, \ldots, M_k = L$ (each M_{i-1} the main parent of M_i), the *epoch* of M_i is the set of blocks it adds to the past cone. The canonical order is

$$\mathrm{epoch}(M_i) = \mathrm{past}(M_i) \setminus \mathrm{past}(M_{i-1}), \qquad \mathrm{Ord}(L) = \mathrm{topo}(\mathrm{epoch}(M_1)) +\!\!+ \cdots +\!\!+ \mathrm{topo}(\mathrm{epoch}(M_k)),$$

with topo a Kahn sort breaking ties by block number, then hash. The order partitions the past cone, respects ancestry, and is prefix-stable along a main chain. If every LFB lies on the main chain of every later LFB (assumption A-LIN), then nodes with different finality step sequences execute the same order. The finalizer does not check A-LIN today; the linearizer must check it and hold on a violation, reporting it as a consensus fault.

The linearizer hooks into the finalizer's existing effect closure, which already receives each newly finalized set (`finalization_runner.rs`, around lines 380–390).

5.5 The executor

Every node runs the executor over the canonical order. For each block b it either executes b and publishes a record, or replays b's canonical record.

```
execute(b):                                   -- R0 = executed(prev(b))
  gate phase, at R0, before any execution:
      for each envelope e of b, in block order:
          skip e if it ran before or is outside its validity window
          compute the holds of e (DR-64 linear estimate)
      group envelopes that share a purse (DR-65 groups)
      reject every group whose holds exceed the purse supply at R0
      reject every envelope whose own holds fail            -- no effect, no charge
  run phase:
      start ONE RSpace session at R0
      inject every accepted envelope at once:  {P1}s1 | {P2}s2 | ... | {Pn}sn
```

```
    reduce concurrently until every envelope has quiesced or exhausted its limit
    races are decided by the run; the COMM log records every winner
settle phase:
    settle each envelope from its measured use; refund the remainder
    pay fees to sender(b); run slash records, then close-block work
rec(b) <- R0, gate verdicts, COMM log, settlements, system work, post-root
```

Properties of this executor:

- **No run-to-completion between envelopes.** Envelopes of one block interleave freely. No envelope waits for another to finish, and nothing requires the result to equal a sequential order of envelopes.
- **The gate precedes all execution.** An envelope rejected at the gate never enters the runtime and pays nothing, as the papers require.
- **Simultaneous arrival.** Envelopes of one block arrive together. Within a group sharing a purse, either the purse covers every member's hold or every member is rejected.
- **Arrival in sequence across blocks.** The canonical order defines "in sequence". A later block's envelope sees the purse as the earlier block left it.
- **Holds live within one block's execution.** No reservation table survives across blocks; the rule against escrow is kept.
- **The cost-cursor bucket locks stay.** DR-119's 256 bucket locks replace the global cost-cursor lock; without them every pair of concurrent settlements would collide on one channel.

Block-level sequencing in the first version. Blocks execute one after another in canonical order: b starts from the root that $\mathrm{prev}(b)$ left. Concurrency in the first version lies inside each block, which carries many envelopes once the one-offer limit is gone. Concurrency across non-conflicting blocks is the refinement of §5.12.

5.6 The order-independent charge model

Replay verifies a record by forcing its recorded communications. For every honest replay of one record to reach the same root *and the same charges* without a canonical schedule, every charge must be a function of the recorded event and its recorded inputs, never of incidental runtime state such as the live size of a map.

1. **Charges as functions of events.** Each host-work charge is restated as a function of the event's own data: the sizes of the terms, patterns and data it touches, and fixed per-operation index charges. Charges that today read live container sizes are replaced by fixed charges bounded by the protocol-6 host-work caps. The first target is the COMM-lookup charge in `replay_authority.rs` (DR-79, C14), which reads the live sizes of three identity maps.
2. **Names by source and occurrence.** An operation is named by its RSpace source (channel hash, produce or consume hash, persistence) and by the number of earlier operations with that source, not by a causal path that depends on which side completed a COMM (the branch's RD-2).
3. **Recorded decisions.** Decisions that depend on interleaving, such as a skip, filter or denial, are recorded in the log and checked on replay, as the branch's budget trace already does (RD-3).

4. **Concurrent replay.** Replay starts each recorded COMM as soon as its recorded predecessors (the produces and consumes it consumes) are present, as `dev`'s `ReplayRSpace` does. It does *not* wait for journal order. Replay is therefore as concurrent as play.
5. **Bounded verdicts.** A forged record that names a predecessor which never appears must end in an "invalid record" verdict, not an unbounded wait. Today native replay waits without a bound (`trace/replay.rs:422-436`); a per-record step bound fixes this (RD-4).

Decision records affected. The scheduler-based records DR-74 and DR-84 to DR-86 (operation naming by causal path) are superseded. DR-78, DR-79 and DR-100 keep their charges but lose their determinism premise and are amended to state charges as event functions. `deterministic_reduction.rs` leaves the v6 path. Dev's file-I/O work is porting that scheduler (Mike Stay's slices, `park_external_during`); that port must be coordinated so that file-lock waits are handled without a frontier barrier.

Known determinism gaps. Bugs 10814 and 10812 let host work vary across hosts and processes. Today's validator replay has the same gap. Both must close before v6 activation, because a verdict at a cap boundary decides consensus.

5.7 Records, canonical records and attestation

The record of one run of block b: pre-root, gate verdicts, the COMM log with per-participant charges, settlements, system work, and post-root. Every item exists today in play's output.

Who runs. By convention the sender of b executes b first, as today's producer plays its own block, and carries $\mathrm{rec}(b)$ in its next block. If no block cites a record of b within k main-chain steps after b is finalized, any validator may execute b and cite its record. The convention avoids duplicate work; consensus does not enforce it.

The canonical record $\mathrm{rec}^*(b)$ is the first record of b, in canonical order of the finalized blocks citing it, whose pre-root equals $\mathrm{executed}(\mathrm{prev}(b))$ and whose replay reaches its stated post-root. A record built on another pre-root is stale and carries no offense. A record whose replay fails or ends elsewhere yields signed invalid-record evidence against the citing sender. Then

$$\mathrm{executed}(G) = r_G, \qquad \mathrm{executed}(b) = \mathrm{post}(\mathrm{rec}^*(b)) = \mathrm{replay}(\mathrm{rec}^*(b)).$$

Why honest nodes agree. They agree on Ord (A-LIN); records travel inside blocks, so record availability is block availability; replay of a record is a function of the record (§5.6); by induction along Ord they agree on every $\mathrm{executed}(b)$.

Attestation. Each block attests (E, r_E). When a node fixes $\mathrm{executed}(E)$, it checks every received attestation of E and records misattestation evidence where they differ, and checks the block's envelopes against the inclusion filter at r_E. A large stake attesting a different root is also an alarm for the node's own replay.

Lag. A record becomes canonical only once a citing block is finalized, so executed state trails the LFB by roughly one execution plus one finality round. That is acceptable for a first version. The refinement of §5.12 shortens it.

5.8 The inclusion filter

The proposer filters candidate envelopes at its attested root: it skips envelopes already in its past cone or outside their validity window, rejects any whose DR-64 check fails on its own sources, rejects any DR-65 group that would overcommit a purse, and stops at the per-block demand cap. The filter is advisory; the executor's gate decides. Envelopes that a concurrent block's spending makes unfundable are rejected at the gate at no cost to their signers.

5.9 Validator sets, epochs and slashing

Blocks no longer have their own post-states, so bonds come from attested executed state. The validator set of epoch e is the bond set in the executed state at the end of an earlier epoch (lag ≥ 1 epoch; question Q-3). Fork choice and the clique oracle give an equivocator zero weight as soon as the target's view holds the equivocation (Q-4). Slash records execute at their block's position. The demand cap and the backpressure rules below keep execution lag below one epoch, so that a node can always check a block's `bonds` field within bounded time (Q-6).

5.10 Backpressure and finality

Removing execution from the consensus path makes blocks cheap to produce and validate. The DAG can then grow faster than finality, and the clique oracle's cost still grows with the unfinalized depth. The finality half of the loop survives in a consensus-only form unless it is fixed. The following are part of this design, not optional:

1. **Finalizer forward progress.** The finalizer derives the floor in resumable steps under a per-run budget. A budget that runs out keeps the cursor as a resumption token and decides nothing. A candidate is adopted only when the derivation is complete *and* a re-validation under the commit-time snapshot passes. The adoption is one atomic transaction. Section 5.11 gives the design. (Revision 1 said that a run at its deadline adopts the highest candidate proven so far. That rule is withdrawn.)
2. **Proposal pacing.** A validator proposes only when it has something new: envelopes, records to cite, or a liveness need. Deploy-triggered requests coalesce into one pending proposal. Above a floor-distance threshold (well below 256), proposers include no envelopes and propose only to advance finality.
3. **Demand cap and execution-lag gate.** Each block's $\sum$ phlo limit is capped. If a node's executed root trails its LFB by more than a threshold, its proposals carry no envelopes until it catches up.
4. **Admission cap.** The deploy pool rejects new envelopes with a retryable "busy" status above a size threshold.
5. **Work off the async runtime.** Execution, replay and checkpointing run on a dedicated runtime or blocking pool, never on the workers serving gRPC and peer traffic.

5.11 Finalizer forward progress

This section replaces item 1 of §5.10 in revision 1. It follows the response of the F1R3node Casper Working Group, "A Live-and-Safe Finalizer-Progress Design" (October 16, 2026), and adds two

points from the review of that response.

Why the rule of revision 1 is withdrawn

The rule "adopt the highest candidate proven so far" treats an oracle verdict on one candidate as a finality decision. The response gives three counterexamples.

1. **Incompatibility under $\theta \leq 0$.** Two disjoint half-stake cliques can certify incompatible blocks of the same height without equivocation. A complete derivation returns `IncompatibilityHold`. A run that certifies A and stops before it examines B finalizes A. A peer with the opposite order finalizes B.
2. **State containment.** After the oracle verdict, `floor_of_view_metered` checks that the derived floor contains every effect settled under the current LFB. A run that stops between the two steps can adopt a floor that regresses settled state.
3. **Race before commit.** Between `Advance(F)` and the metadata write lock in `record_directly_finalized`, new latest messages can arrive. Only the membership of F in the DAG is checked again.

The rule also has no progress argument. If every run stops before it certifies one candidate, there is nothing to adopt.

Review notes. Counterexamples 1 and 2 refute the rule of revision 1. The delivered fix A1a does not use that rule. A timed-out run still discards its derivation, and only a complete derivation can advance the LFB. Counterexample 3 applies mainly under $\theta \leq 0$. The finalizer is the only writer of the LFB in steady state, so the LFB cannot move between derivation and commit. Under $\theta > 0$, a new latest message cannot undo a valid certificate. The re-validation below costs little, so the design adopts it in all regimes. The review found one more defect in `dev`: the 15 s timeout wraps the commit effect as well as the derivation (`finalization_runner.rs` on `dev`, lines 278 and 602). The effect has an await point between its snapshot and its metadata write, so a timeout can cancel a commit part of the way through. Fix A1b closes this before A1c lands.

Three phases

Compute
A resumable state machine with a bounded budget. A step returns `Yielded(cursor)` or `Terminal(FloorOfView, σ)`. Compute never commits. The cursor states follow the phases of `floor_of_view_metered`: tips, decidability, candidates, the oracle sweep (one oracle call is one step), parent floors, the compatibility pairs, and containment. The budget is checked only between steps. Tests use a step-count budget for determinism. Production uses wall-clock time.

Validate
Runs while it holds the metadata write lock, under the current snapshot σ'. It discharges the preconditions P1–P4 below.

Commit
One LMDB write transaction across the block metadata, a new finalization commit marker and a one-row LFB pointer. The transaction uses `batched_put` across stores that share one environment. The in-memory DAG state changes after the transaction commits, under the same guard.

A deadline fires between Compute steps only. It never cancels Validate or Commit.

Snapshot identity and the change policy

The snapshot identity σ is a 32-byte Blake2b hash of: the sorted pairs (validator, latest message hash), the current LFB hash and height, the threshold θ, and a hash of the equivocation-tracker state. Bonds and epoch are functions of the LFB, so the LFB hash covers them.

Memos are in two layers:

- **Facts about immutable blocks.** These survive a change of σ. Examples are self-justifications, justification maps keyed by message, main-chain membership keyed by (target, block), the frontier index, the fault-tolerance lower bound and the lineage memos. Fix A1a delivered the oracle part of this layer: one shared cache, capped at 200,000 entries per map.
- **Work that read the snapshot.** Tips, candidate selection and verdicts keyed by (target, σ). A change of σ discards this layer and resets the cursor to the tips state.

Anti-livelock. If changes of σ invalidate N consecutive runs (default 4), the finalizer freezes to the last stable σ for one run. The result is still safe, because Validate checks it under σ'.

Preconditions at commit

P1 The candidate F is in the DAG.

P2 The state of F contains the state settled under the current LFB. This closes counterexample 3.

P3 Under $\theta > 0$, no `IncompatibleFinalizedFork` among the siblings of F in the current snapshot. This is a loud alarm, not a hold. Under $\theta \leq 0$, `IncompatibilityHold` halts the commit. This closes counterexample 1.

P4 A-LIN: F is a main-parent-chain descendant of the current LFB. A violation halts with `ALinViolation`. Question Q-1 becomes an observable alarm.

If a precondition fails, the commit aborts with `StateChanged` and the next cycle derives again. One bounded retry is permitted. The 128-iteration reconcile loop in `record_directly_finalized` is removed.

Events and crash recovery

Events publish after the transaction commits, in sorted (block number, block hash) order. Today they publish in `HashSet` order. A new event variant, `BlockFinalisedV2`, carries a `commit_epoch`. The variant is new because `bincode` does not accept trailing fields. At startup, `reconcile_pending_finalization` reads the marker. A marker with `events_published=false` causes the events to publish again with the same `commit_epoch`. A stale metadata flag without a marker is corruption and halts the node.

Liveness

The response states the assumptions: eventual synchrony, an honest-stake supermajority suitable for the regime ($\geq 2/3$ under $\theta > 0$), fair block propagation, and a per-run budget large enough for one

Compute step. Under these assumptions:

L1. Facts about immutable blocks accumulate across runs and are never invalidated.

L2. After the global stabilization time, σ stays stable for one cycle of work.

L3. Under a stable σ, the cursor reaches `Ready` in $O(|\text{frontier}|)$ steps.

L4. Validate is one cheap step. In the worst case the cursor restarts under σ' and L1 keeps the per-block work.

L5. Commit is atomic with respect to crashes and to concurrent runs.

Points added by the review

1. **Asynchronous finalization effects.** Today `record_directly_finalized` runs an asynchronous effect outside the lock: deploy-store cleanup and runtime-manager work. That effect is the reason for the reconcile loop. The signature of `commit_lfb` does not include it. This design runs the effect after the commit, keyed to the commit marker, and requires it to be idempotent (Q-12).
2. **The freeze is the main lever under load.** Under load, almost every block changes some latest message, so σ changes in almost every run. Windows with a stable σ are therefore rarest in the stall regime. `dev` already derives each run on a frozen snapshot. The design makes the freeze the default when the invalidation rate is high, not only after N failures, and measures it with `finalizer_cursor_invalidations_total`.

Effect on v6

P4 checks at commit the assumption that the v6 linearizer relies on. The linearizer hook (§6.4) takes the newly finalized set from the post-commit publication. That set is in sorted order and is published again after a crash, so the linearizer sees each finalized set exactly once per `commit_epoch`.

5.12 Signature footprints and the partial-order refinement

By decision 2, every potential conflict is marked by signature. Define the *signature footprint* $\Sigma(b)$ of block b as the set of signatures whose pools any communication in $\mathrm{rec}^*(b)$ draws on, both sides of every COMM included. Two blocks b, b' commute when $\Sigma(b) \cap \Sigma(b') = \emptyset$. The footprint is read from the record; computing the intersection is a set operation, not a merge.

Refinement R1: concurrent execution of non-conflicting blocks. Within one main-chain epoch, the executor runs blocks with disjoint footprints concurrently and composes their state changes; blocks that conflict run in canonical order. Because the footprint is known only after a run, the executor runs speculatively and re-runs a block whose realized footprint meets an earlier concurrent block's. Replay is unaffected: it follows records.

Refinement R2: speculative execution on the fork-choice chain. The sender of b executes b on its fork-choice view at proposal time. If the canonical pre-root later matches, the record stands; otherwise b is re-executed. This removes most of the lag of §5.7.

Both refinements depend on one property to be verified before they ship: that the runtime's per-participant charging marks *every* interaction, including a consume that wins a datum produced under

a third signature. A property test over recorded COMM logs checks that every COMM's participants all appear in the footprint.

5.13 How the design addresses the loop

Loop component	Under this design (v6)
Merge scope and rejection search	Gone. Blocks carry no post-state, so there is nothing to merge.
Validation replay on the consensus path	Gone. Validity is structural; replay happens behind finality.
Pool re-reads and lineage walks per deploy	The inclusion filter reads one attested root; the lifecycle register is replaced by the gate. Pool indexing and the admission cap remain required (Track A).
Finality oracle growth and timeout trap	Not removed by execution changes. Fixed by §5.10 items 1–3 and the finalizer design of §5.11.
Hot-channel copies	Priced by host-work charges; representation fix in Track A.
Runtime starvation	Execution and replay run off the async runtime.
Floor-distance cliff	No merge, so no cliff; pacing keeps distance bounded.

5.14 Properties to verify

Property	Kind	Meaning
ExecutedRootAgreement	safety	Honest nodes that fix $\mathrm{rec}^*(b)$ set the same $\mathrm{executed}(b)$.
ReplayAgreement	safety	Any two honest replays of one record, in any order consistent with its recorded dependencies, reach the same root, charges and verdict.
OrderIndependentCharge	safety	Every charge is a function of the recorded event; permuting causally independent COMMs leaves every charge unchanged.
GateBeforeExecute	safety	No step of an envelope runs before its gate accepts it.
AtMostOneOfCompeting	safety	Of two envelopes a purse cannot fund together, at most one settles.
ValidationNeverExecutes	safety	Structural validation runs no Rholang and replays no record.
FootprintCommutation	safety	Blocks with disjoint signature footprints commute (needed for R1).
BoundedLag	liveness	Under the demand cap and pacing, every finalized block gets a canonical record within bounded distance of the LFB.

Property	Kind	Meaning
BoundedReplayVerdict	liveness	Replay of any record, including a forged one, ends with a verdict.
Finalizer (§5.11); the response requires these tests before A1c is accepted.		
NoConflictingFinality	safety	Two nodes with reversed candidate orders on an evenly split committee both reject with `IncompatibleFinalizedFork` under $\theta > 0$ and both hold under $\theta \leq 0$.
NoSettledRegression	safety	A certified candidate whose state omits a settled effect makes the commit abort with `StateChanged`. The cursor is kept.
IncompleteProofNever-Commits	safety	A deadline injected at every cursor state never reaches `Ready` and never calls `commit_lfb`.
CacheTransparency	safety	Cached and uncached derivations give the same `FloorOfView`. Fix A1a tests this across seven snapshots.
CanonicalOrderSafety	safety	A candidate off the main-parent chain of the current LFB halts the commit with `ALinViolation`.
CommitRevalidation	safety	A latest message injected between `Ready(Advance(F))` and the write lock makes the re-validation reject. The next cycle derives again.
AtomicAdoption	safety	A fault between the marker write and event publication causes a replay with the same `commit_epoch`, and consumers see the event once.
ResumableLiveness	liveness	A deterministic step budget forces at least 10 resumptions on a 50-validator network, and the cursor reaches `Terminal(Advance(_))`.

The 60 h operational soak adds the metrics `finalizer_cursor_yields_total`, `finalizer_cursor_resumes_total`, `finalizer_cursor_invalidations_total` and `commit_lfb_toctou_rejections_total`. They must stay bounded while the LFB advances. The soak is operational evidence, not a substitute for the tests above.

TLA+ models must include failing configurations that refute each safety property when its guard is removed (for example, a model where replay reads live sizes must refute OrderIndependentCharge).

6 Development plan

6.1 Status (October 16, 2026)

Work	Branch	Content and state
A1a, A2, A3	`fix/finality-stall-track-a` (three commits on `dev`)	A1a: the oracle cache keeps facts about immutable blocks across runs and snapshots; a timed-out run starts again at once, at most three times in a row. A2: deploy-triggered proposals coalesce and do not mint empty blocks. A3: proposals carry no user deploys when the floor distance exceeds 128; the pool refuses deploys above $64\times$ `max-user-deploys-per-block` with a retryable error (HTTP 503 with `Retry-After`). Pushed; pull request pending.
v6 tranche 1	`feature/cost-accounted-ote` (five commits on `dev`)	The v6 consensus seam of B4 behind the flag `v6-order-then-execute` (default `false`): proposal without execution, structural validity, the linearizer, records, attestation and evidence. All envelopes of a block run concurrently in one RSpace session, so cross-deploy races are real. Limits: one budget and one deploy-data cell per block until B1; funding gate and settlement are stubs; evidence is logged, not slashed; executor state is in memory. Pushed; pull request pending.

6.2 Principles

- **Legacy path byte-identical to `dev`.** Every increment merged to `dev` must leave legacy roots and legacy replay unchanged on a recorded corpus, and legacy performance within 3 % on the reference tests.
- **Casper changes only at hook points.** The v6 consensus seam lives in a new module, `casper/src/rust/v6/`, entered from four hook points in `dev`'s Casper code (§6.4).
- **Leave behind what the design makes unnecessary.** The interim v6 merge rule, the one-offer block lane, the G2 ordered batch executor, offered recovery through the rejected buffer, and the scheduler are not merged.
- **Small, reviewable increments.** Formal artefacts and design documents go in their own pull requests, separate from code.
- **Two tracks in parallel.** Track A fixes the stall on `dev` now; Track B lands cost accounting.

6.3 What is kept, ported, and left behind

Branch content	Disposition	Notes
Rholang `accounting/`, metering, host-work, matcher, substitute, compiler changes	Keep	Core of B1; gated so legacy evaluation is unchanged.
RSpace native store and native replay session	Keep, then revise	Charges rewritten as event functions in B2.
Models: offered envelope, protocol limits	Keep	Additive protobuf fields only.
`SystemVault.rho` bucket locks (DR-119)	Keep	Needed for concurrent settlement.
Funding gate G1 (DR-64, DR-124), DR-65 grouping	Keep	Becomes the executor's gate phase.
Installer funding G3 (DR-68)	Defer	Not required to break the loop.
`deterministic_reduction.rs` scheduler	Leave	Superseded by the charge model (decision 1).
v6 merge rule (DR-120, `costacc/v6_merge/`)	Leave	No merge under order-then-execute.
G2 ordered batch executor, one-offer lane in `block_creator.rs` (+1,581 lines)	Leave	Replaced by the concurrent executor and v6 proposer.
Offered recovery (DR-116), rejected-buffer changes	Leave	A gate rejection has no effect; clients resubmit.
`formal/` (184k lines), `docs/casper/theory` (63k lines)	Separate PRs	Reviewed independently of code; superseded models marked.

6.4 Hook points in the Casper code of dev

Hook	Location in `dev`	Change
Propose without execution	`block_creator.rs`, block creation entry	If the shard is v6, call `v6::propose` (inclusion filter, attestation, records) and return.
Structural validity	`validation_dispatcher.rs`, the call to `validate_block_checkpoint` (about line 145)	If v6, call `v6::validate` instead of the checkpoint replay.
Linearizer	`finalization_runner.rs`, the new-LFB effect closure; after A1c, the post-commit publication	Emit the newly finalized set to `v6::linearizer`, in sorted order, once per `commit_epoch`.

Hook	Location in `dev`	Change
Block body	`CasperMessage.proto`, `BodyProto`	Additive `records` field.

Each hook is a branch on one predicate, `is_v6_shard()`, read from the adopted genesis resource policy. Estimated change to `dev`'s Casper files: a few hundred lines; everything else is new code.

6.5 Track A: stall fixes on dev

These fixes are independent of cost accounting. Items A1–A3 are also prerequisites for v6, because they address the finality half of the loop. Revision 2 splits A1 into three parts, after §5.11.

ID	Change	Exit criterion	Size
A1a	*Done.* One oracle cache of facts about immutable blocks, shared across runs and snapshots; a timed-out run starts again without a trigger, up to three times. No partial result is committed.	Frontiers equal with and without the cache across snapshots; timeout-rerun tests.	S
A1b	Move the commit effect outside the 15 s deadline. Only the derivation is timed.	Test: a timed-out run never commits; a commit is never cancelled.	S
A1c	Compute / Validate / Commit (§5.11): `FloorCursor` and the step function; snapshot identity σ and the change policy; `commit_lfb` with P1–P4 in one LMDB transaction; commit marker and startup reconciliation; `BlockFinalisedV2`; sorted event order; no reconcile loop. P4 can land first, alone.	The finalizer tests of the properties table pass; `finalizer.run.timeouts` need not be 0, but the LFB never stalls in a 60 h soak.	L
A2	*Done.* Coalesce proposals; deploy-triggered proposals never mint empty blocks. Debouncing the heartbeat remains.	Proposer queue depth ≤ 1; empty-block rate falls.	S–M
A3	*Done.* Floor-distance pacing above 128 (half the cap of 256); admission cap on the pool with a retryable "busy" status. Thresholds are constants; moving them to configuration remains.	Floor distance bounded under burst load.	S
A4	Pool lock narrowed to snapshot-and-release; in-memory pool index; no signature re-verification on reads from the node's own store.	Prepare time flat in pool size.	M

ID	Change	Exit criterion	Size
A5	Lifecycle register batched per floor advance using cached `block_facts`.	Observe time flat in open signatures.	M
A6	Replay, merge and checkpoint moved to a dedicated runtime or blocking pool.	No consensus task on tokio workers.	M
A7	Fix buffer eviction order; keep re-fetch cooldown outside `ack_*`.	Unit tests; re-fetch traffic linear.	S
A8	In-memory DAG indices: `contains`, latest messages, invalid blocks, equivocation set; read-only equivocation access under the read lock.	No full-table scan per block.	M
A9	Cap and simplify the legacy rejection search (deterministic greedy fallback; stop unioning groups by mergeable channel).	Bounded merge time on legacy shards.	M

Sizes: S is up to 1 engineer-week, M 1–3 engineer-weeks, L 3–8, XL more than 8. All sizes are estimates.

6.6 Track B: cost-accounted order-then-execute

B0. Integration branch and baseline (S). Cut `integration/cost-accounted-ote` from current `dev`. Record the legacy reference corpus (block roots and replay roots of the multi-parent test suite) and the legacy performance baseline (tests T1–T3 of the branch proposal). Move `formal/` and design documents into separate PRs.

B1. Core accounting into `dev` (L). Port the Rholang accounting core, the RSpace native store and replay session, the models and shared utilities. Resolve the 38 conflicts, concentrated in `reduce.rs`, `rho_runtime.rs`, `errors.rs`, `host_work.rs`, `rholang_merging_logic.rs`, the RSpace history and radix files, `CasperMessage.proto`, `casper_message.rs`, and the Casper runtime files. The scheduler is not ported; the legacy evaluation path stays `dev`'s racing evaluation. *Exit:* all `dev` tests pass; the legacy corpus is byte-identical; legacy performance within 3 %; offered unit tests pass in isolation. *First task:* establish whether the core builds without the branch's Casper plumbing in `runtime.rs` and `runtime_manager.rs`; if not, extract the minimum plumbing behind `is_v6_shard()`.

B2. Order-independent charge model (L). Restate every host-work charge as an event function (§5.6); name operations by source and occurrence; record interleaving-dependent decisions; make native replay concurrent with a bounded verdict. Supersede DR-74 and DR-84–86; amend DR-78, DR-79 and DR-100. Close bugs 10814 and 10812. *Exit:* property test replaying each recorded block under 256 random causal-order permutations yields identical roots, charges and verdicts; OrderIndependentCharge model-checked with its failing configuration; Rocq lemma that charges are invariant under swapping independent COMMs.

B3. Concurrent block executor (L). Implement `v6::executor` as a library: gate phase, one concurrent session per block, settlement, record. Test it offline against the offered test corpus before

any Casper wiring. *Exit:* executor records replay to identical roots across hosts; GateBeforeExecute and AtMostOneOfCompeting hold under property tests with shared purses and shared data channels; throughput scales with envelopes per block.

B4. v6 consensus seam (XL). *Tranche 1 delivered (§6.1).* Implement `v6::propose`, `v6::validate`, `v6::linearizer`, the record field, and the four hooks. Check A-LIN in the linearizer, and at commit through P4 of A1c. *Exit:* a three-validator v6 shard runs end to end; ValidationNeverExecutes holds; legacy corpus still byte-identical.

B5. Records, attestation and evidence (L). Canonical-record selection, the runner convention and fallback, attestation checks, invalid-record and misattestation evidence, record retention. *Exit:* ExecutedRootAgreement and ProvableEvidence model-checked with three validators and permuted arrival; fault-injection tests with forged records.

B6. Epoch validator sets and slashing timing (M). Bonds from attested state with epoch lag; immediate zero weight for equivocators in fork choice and the oracle. *Exit:* TLA+ check that safety holds under the lagged weight source; Casper team sign-off on Q-3, Q-4, Q-6.

B7. v6 soak and activation (M). Three- and six-validator v6 soaks with the existing stage histograms, against a legacy soak on the same hardware. *Exit criteria for activation:* no stall in a 60 h soak; finalization p95 within the soak gate; executed-root lag bounded; host-work caps settled (the Embers log initializer currently exceeds them); settlement log-channel continuations metered.

B8. Refinements (L each, after activation). R1, concurrent execution of non-conflicting blocks by signature footprint; R2, speculative execution on the fork-choice chain.

6.7 Sequencing

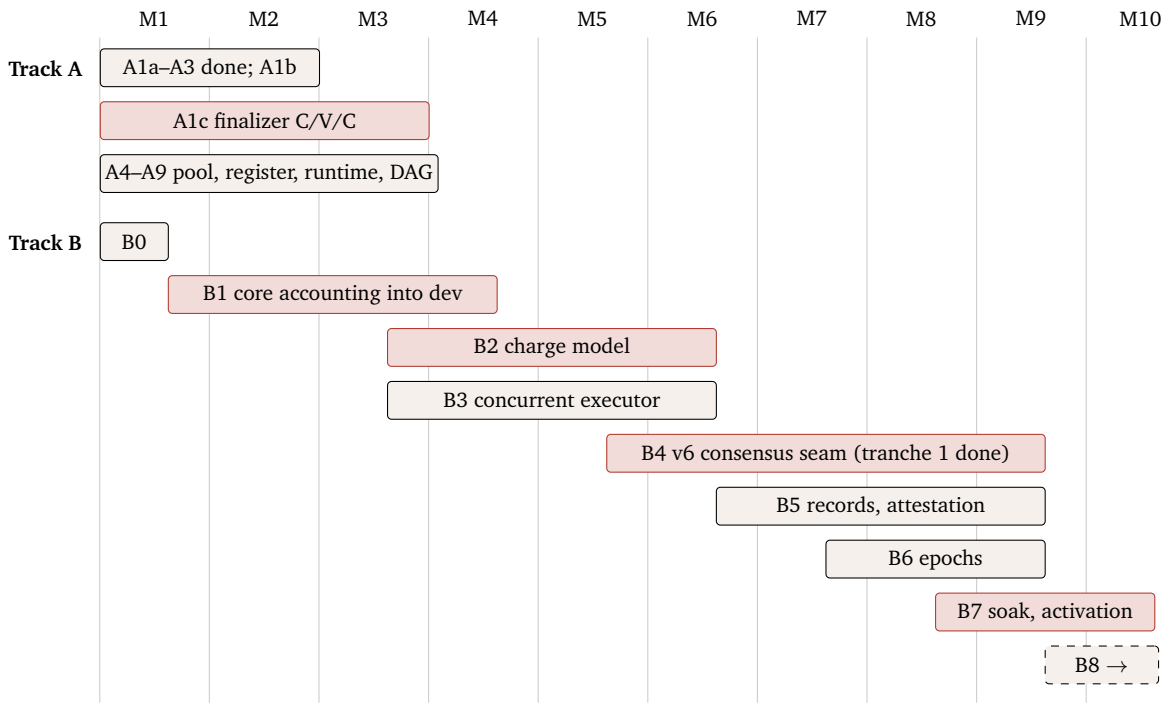


Figure 3: Indicative sequencing in months, assuming three to four engineers on Track B and the Casper team on Track A. Shaded bars are on the critical path. Estimates, not commitments.

B2 and B3 can run in parallel once B1's core is in `dev`. B4 depends on B3's library interface and on A1–A3, because a v6 shard without the finality fixes would reproduce the consensus half of the loop. A1c is on the critical path: B7 activation needs it. Its precondition P4 should land early, because it checks A-LIN, which the v6 linearizer relies on. Tranche 1 of B4 ran ahead of B1–B3 by sharing one budget per block. The remaining B4 work picks up per-signature budgets when B1 lands. The overall estimate for B0–B7 is about ten calendar months with three to four engineers on Track B. For comparison, the branch proposal estimates its order-then-execute phase alone at 4–9 engineer-months; this plan adds the core merge (B1), the charge model (B2) and the backpressure work.

6.8 Ownership

- **Casper team:** Track A, with the Casper Working Group owning A1c; the hook points of B4; B5 evidence classes; B6; answers to Q-1 to Q-7.
- **Cost-accounting lane:** B1, B2, B3, and the `v6` module of B4.
- **Formal methods:** the models and proofs listed with each exit criterion, and marking superseded scheduler-based models.
- **File-I/O team:** coordination on the scheduler port in `dev` (decision 1).

7 Risks and open questions

7.1 Risks

Risk	Mitigation
A-LIN fails (an LFB off a later LFB's main chain)	Linearizer checks and holds; the commit halts with `ALinViolation` (P4); Casper team confirms A-LIN from the clique oracle's spine argument, or documents the halt as the canonical response (Q-1).
A deadline cancels a commit part of the way through (`dev` today)	A1b times only the derivation; A1c makes the commit one atomic transaction.
Snapshot churn under load leaves no stable window	Freeze to the last stable snapshot, by default when the invalidation rate is high; Validate keeps the result safe; the invalidation metric is part of the soak gate.
Atomic commit changes the storage schema and the event wire format	New marker store and LFB pointer, included in soak snapshots; `BlockFinalisedV2` as a new enum variant; startup reconciliation; consumers already deduplicate or forward `commit_epoch`.
Host-work nondeterminism across hosts (bugs 10814, 10812)	Close in B2; cross-host replay in B3's exit criteria.

Risk	Mitigation
Finality remains the bottleneck after execution leaves consensus	A1–A3 are prerequisites for B4; pacing and lag gates in the design.
Core accounting cannot be separated from Casper plumbing	First task of B1; minimal plumbing behind `is_v6_shard()` if needed.
`dev` keeps moving (470 commits since the merge base)	Rebase B1 weekly; land in small PRs.
Scheduler port on `dev`'s file-I/O line conflicts with decision 1	Agree with the file-I/O team before B1 lands.
Record size and retention	Per-block record cap; retention policy (Q-7); state sync from attested roots (Q-5).
Footprint marking incomplete for some interaction	Property test in §5.12 before R1 ships; R1 is not on the critical path.
Client changes	Clients size phlo limits to the linear estimate; Embers and the gateway need updates before activation.

7.2 Open questions for the Casper team

ID	Question
Q-1	Does every LFB lie on the main chain of every later LFB (A-LIN)?
Q-2	Which evidence class and penalty apply to misattestation, invalid records and inclusion-rule violations?
Q-3	How many epochs of lag separate the attested bond state from the epoch that uses it?
Q-4	May fork choice and the clique oracle give an equivocator zero weight as soon as the view holds the equivocation?
Q-5	How much attesting stake makes a root acceptable for state sync?
Q-6	How long may a node hold a block whose `bonds` field it cannot yet check?
Q-7	Runner fallback distance k, per-block record cap, and record retention.
Q-8	Threshold values: floor distance for pacing, executed-root lag for the envelope gate, pool admission cap. A3 sets defaults of 128 blocks and $64\times$ the per-block deploy limit.
Q-9	Should the finalizer cursor survive a restart? The response proposes memory only for a first version.
Q-10	May `propagate_ft_to_finalized_blocks` stay outside the atomic commit? It raises a monotone lower bound, so a partial crash heals itself.
Q-11	Is `commit_epoch` counted per LFB or per boot? The response recommends per LFB.

ID	Question
Q-12	Where do the asynchronous finalization effects run once `commit_lfb` is atomic? This design proposes after the commit, keyed to the marker, and idempotent.

A Detailed stall findings

The table lists the findings of the stall review beyond those in Section 2, by subsystem, with the lines at which each was located on `master` at `7264c98`. Severity is the review's estimate of the contribution to a stall that worsens under load.

Subsystem	Finding	Location	Sev.
Proposer	Unfinalized cone walked once per parent with full block decodes.	`block_creator.rs:1497-1725, 2085-2131`	High
Proposer	Whole pool sorted and encoded per selection, three times.	`block_creator.rs: 269-279,349-415`	Med
Proposer	Per-deploy WARN logged for every in-scope deploy on every proposal.	`block_creator.rs:1380-1440`	Med
Merge	Retry loop rebuilds branches and conflict map each pass.	`dag_merger.rs: 705-752`	High
Merge	Stale-diff expansion runs one DAG search per block pair.	`dag_merger.rs: 1855-1915`	Med
Merge	Dependency ordering cubic in branch size; `depends` allocates sets per call.	`dag_merger.rs: 82-120,1456-1502`	Med
Merge	`ChannelChange::combine` quadratic on hot channels.	`channel_change.rs:17-55`	Med
Merge	Block-index cache returns deep clones; bypasses its FIFO bound.	`interpreter_util.rs: 1316-1342`	Med
Finality	Uncached LMDB metadata lookups inside `block_in_place`.	`block_dag_key_value_storage.rs:144-151`	High
Finality	FT propagation rescans up to 50,000 blocks under both write locks.	`block_dag_key_value_storage.rs:1699-1727`	Med
Finality	Estimator scoring walks $O(V \times (\text{tip} - \text{LCA}))$.	`estimator.rs: 232-378`	Med
Finality	Redundant fault-tolerance oracle run after each LFB advance.	`finalization_runner.rs: 571-576`	Med

Subsystem	Finding	Location	Sev.
Block processing	DAG membership via full representation per gossip message.	`block_admission.rs:34-67`	High
Block processing	Pendant full scan every 32 blocks with block decodes.	`buffer_resolver.rs:59-109`	Med
Block processing	Retriever sends serially, one hash at a time.	`block_retriever.rs:1032-1139`	Med
Block processing	Full snapshot computed per incoming block; cache keyed on DAG generation.	`block_processor.rs:548-553`	Med
RSpace	`put_continuation` formats a `Debug` string per waiting continuation.	`hot_store.rs:528-559,1000`	Med
RSpace	Checkpoint CPU and LMDB writes on tokio workers.	`ispace_impl.rs:41-78`	Med
RSpace	History caches cleared in bulk; radix cache evicts arbitrary nodes.	`hot_store.rs:1091-1117`, `radix_tree.rs:875-890`	Med
Node	gRPC `find_deploy` server-side retry loop.	`deploy_grpc_service_v1.rs:570-643`	Med
Node	HTTP offload without an admission limit.	`shared_handlers.rs:626-640`	Low

B Sources

- `F1R3FLY-io/f1r3node-rust`, branches `master`, `dev` and `feature/cost-accounted-rho` at the commits listed on the first page.
- `docs/casper/theory/cost-accounting-consensus-proposal.md` on `feature/cost-accounted-rho` (amended October 13, 2026): Greg Meredith's four expectations, the paper quotations, the decision records DR-64 to DR-124, the soak and profile data, and the proposal's §7.
- The cost-accounting papers in the F1R3FLY publications repository, as quoted in that proposal. This document did not read the papers directly; quotations are as the proposal gives them.
- `docs/soak-benchmarks.md` on `master`, for the soak lifecycle and gates.
- F1R3node Casper Working Group, "A Live-and-Safe Finalizer-Progress Design: Response to Stall-Loop-Design §5.10.1", October 16, 2026, `F1R3FLY-io/publications`, `cost-accounting/stall-loop-design-5-10-1-response.tex`.
- Branches `fix/finality-stall-track-a` and `feature/cost-accounted-ote` on `F1R3FLY-io/f1r3node-rust`, as pushed on October 16, 2026.